GPAS: OPTIMIZER-AWARE MICRO-BATCH ALLOCATION FOR MULTI-TEACHER ON-POLICY DISTILLATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-teacher on-policy distillation (MOPD) trains one student on its own rollouts, each scored token by token by the specialist teacher for its task. A task mixture is one knob doing two jobs: it sets both objective weight and sample count, even though task-gradient noise differs across tasks and drifts during training. On-policy distillation also entangles objective level and noise inside each token score: the mean student–teacher log-probability gap measures local disagreement, whereas centered variation across sampled tokens can make the score-function gradient noisy and can differ with teacher distance. Gradient-Preconditioned Adaptive Sampling (GPAS) fixes the objective weights, computes each task gradient with an estimator that is exact on the student's top-k tokens and uses the sampled token for an unbiased tail correction, and allocates micro-batches according to the remaining noise in the optimizer's preconditioned coordinates; an optional cost-aware mode targets variance per unit time. In four-teacher distillation of Qwen3-1.7B on mathematics, code, instruction following, and science, **[TODO: task noise spans X across tasks, drifts by Y during training, and the preconditioner changes its ranking in Z of the steps.] [TODO: The token component accounts for X of the noise for distant teachers and Y for same-origin teachers; integrating the retained-token contribution narrows the cross-task spread from A to B, while that integration and allocation together improve matched-step teacher loss by C.] [TODO: At overhead ratio R, the cost-aware mode reaches Uniform's loss in S fewer GPU hours, while no task falls below its Uniform counterpart on downstream benchmarks.]**

1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) merges several specialist teachers into one student: the student generates a response, and the specialist for its task scores that response token by token (Ma et al., 2026). Existing recipes must also choose a task mixture. Token-share balancing changes that mixture to compensate for response-length imbalance (Gao et al., 2026), whereas gap-following changes it as teachers become easier or harder to imitate (Sun et al., 2026). In either case, the same mixture controls what the student learns and how much evidence estimates each task gradient.

A mixture is one knob doing two jobs. Sampling a task more often changes both the objective the student minimizes and how many samples estimate that task's gradient, so any mixture that adapts to training progress changes what is learned and how precisely at the same time. The precision side is not a detail: the noise of a task's gradient differs across tasks, drifts during training, and reaches the optimizer only after it preconditions each coordinate, so the task whose noise dominates a step depends on which optimizer is used.

On-policy distillation weights every sampled token by the difference between the student's and the teacher's log-probability. Averaged under the student, this difference is the local reverse-KL gap; after centering, its variation across tokens interacts with the score function to determine the gradient. At one position, two tokens of similar student probability can carry weights of opposite sign and very different magnitude. That spread creates sampling noise and can differ across teachers because teachers differ in how far they are from the student. The teacher already computes a distribution at

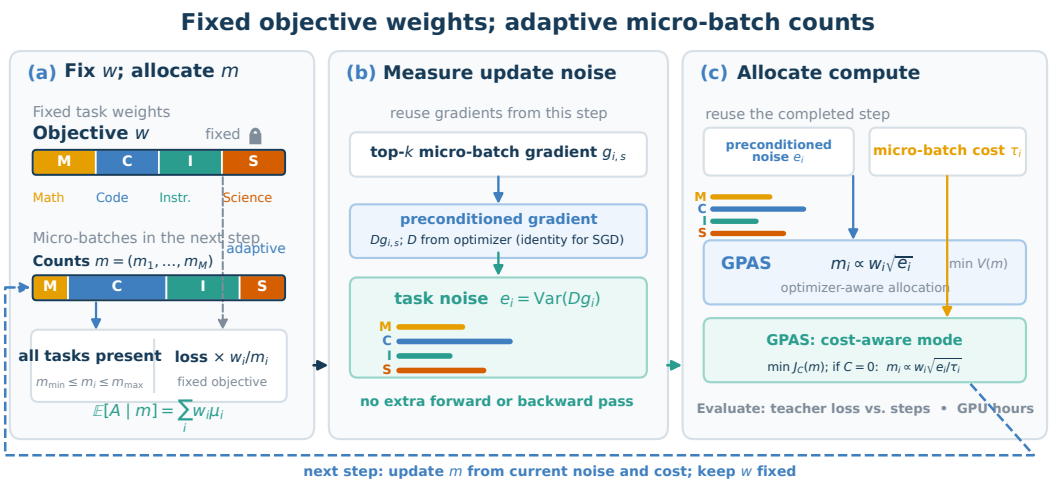

Figure 1: GPAS fixes the objective weights w and adapts only the count vector $m = (m_1, \ldots, m_M)$. Each micro-batch gradient comes from the top-k estimator. The completed step supplies the preconditioned gradient Dg, where D comes from the optimizer and is the identity for SGD; measured time enables the optional cost-aware mode. No additional model pass is required. Segment widths and noise profiles are schematic.

every position, yet the usual sampled-token estimator keeps only one entry. We test whether larger measured gaps predict larger spread rather than assuming that relationship as a theorem.

We keep the objective fixed, compute each task's gradient with an estimator that integrates the top-k token contribution analytically, and spend micro-batches on the noise that remains, measured in the coordinates the optimizer's preconditioner defines.

[TODO: First empirical law: quantify the cross-task range and training drift of preconditioned noise, and report how often its ranking differs from the unpreconditioned ranking.] [TODO: Second empirical law: report how the token component tracks teacher–student gap and teacher origin, and how much the top-k estimator narrows the remaining cross-task spread.] [TODO: Third empirical law: at matched steps, report the separate and joint gains from integrating the retained-token contribution and allocating the residual noise, including the loss-gap control.] [TODO: Fourth empirical law: report the overhead ratio and the GPU hours saved by the cost-aware mode, together with every task's downstream change relative to Uniform.]

Our contributions are:

- a fixed-objective MOPD estimator whose micro-batch counts change variance but not expectation, with a local variance metric defined by any optimizer's pre-step diagonal preconditioner;
- a diagnosis connecting teacher–student gap to token-level noise, a conditionally unbiased top-k estimator that integrates its retained-set component, and a complementary rule that allocates counts using the noise that remains;
- a four-teacher study of Qwen3-1.7B organized around the resulting empirical laws, with matched-step and matched-time comparisons and per-task capability measurements.

2 Preliminaries

2.1 The MOPD objective

There are M tasks, each with a fixed specialist teacher. For a prompt x from task i, the student samples one response y, and teacher i, denoted by π_{T_i}, scores every position. At position t, the conditioning context consists of the prompt x and the student-generated response prefix $y_{<t}$. If y_t is

sampled from the student's next-token distribution, the usual reverse-KL token-gradient estimator is

$$\widehat{h}_i^{\mathrm{samp}}(x, y_{<t}, y_t) = \mathrm{sg}[\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{T_i}(y_t \mid x, y_{<t})] \times \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}), \qquad y_t \sim \pi_\theta(\cdot \mid x, y_{<t}).$$

where sg stops gradients through the log-probability difference. Thus the student is pulled toward the teacher on the tokens it actually produced.

The teacher already computes scores over the vocabulary, and the student already identifies its own high-probability tokens. Let $\mathcal{K}_k(x, y_{<t})$ be the student's top-k set at this context. For a candidate token a, define

$$r_{i,a}(x, y_{<t}) = \mathrm{sg}[\log \pi_\theta(a \mid x, y_{<t}) - \log \pi_{T_i}(a \mid x, y_{<t})] \, \nabla_\theta \log \pi_\theta(a \mid x, y_{<t}).$$

We compute the token gradient as

$$\widehat{h}_i^{\mathrm{top}k}(x, y_{<t}, y_t) = \sum_{a \in \mathcal{K}_k(x, y_{<t})} \pi_\theta(a \mid x, y_{<t}) r_{i,a}(x, y_{<t}) + \mathbf{1}\{y_t \notin \mathcal{K}_k(x, y_{<t})\} \, r_{i,y_t}(x, y_{<t}), \qquad y_t \sim \pi_\theta(\cdot \mid x, y_{<t}).$$

At every fixed prefix this has the same expected gradient as the sampled-token form; it integrates out which retained token was sampled, while the observed token supplies an unbiased correction for the omitted tail (proof in Appendix A.6). It requires no additional teacher or student pass because both distributions used by the expression are already computed. In automatic differentiation, the top-k membership and the coefficients $\pi_\theta(a \mid x, y_{<t})[\log \pi_\theta(a \mid x, y_{<t}) - \log \pi_{T_i}(a \mid x, y_{<t})]$ are treated as stopped constants.

For each task, token contributions are averaged within a response and response means are then averaged within a micro-batch; $L_i(\theta)$ denotes the expectation of that task-level loss. Fixed positive weights, chosen before training and summing to one, define the learning objective

$$F(\theta) = \sum_{i=1}^{M} w_i L_i(\theta). \tag{1}$$

The fixed w_i specify what the student learns, independently of how many micro-batches estimate each term.

2.2 Gradient of a mixed batch

A micro-batch is a group of b prompts from one task, processed in one backward pass. An optimizer step accumulates G micro-batches; task i contributes the integer count m_i, the counts sum to G, and each count lies between $m_{\min}$ and $m_{\max}$. The lower bound is at least two so that a within-task sample variance exists on every step.

Let $g_{i,s}$ be the unweighted gradient of micro-batch s from task i, computed with the top-k estimator above. Multiplying its loss by w_i/m_i after the token and response averages gives the accumulated gradient

$$A = \sum_{i=1}^{M} \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s}. \tag{2}$$

For fresh micro-batches that are conditionally independent across draws and identically distributed within each task, with a common task mean and covariance, let $\mu_i(\theta) = \mathbb{E}[g_{i,s} \mid \theta]$ and $\mu(\theta) = \sum_i w_i \mu_i(\theta)$. Every feasible allocation satisfies $\mathbb{E}[A \mid m, \theta] = \mu(\theta)$, so changing the counts changes precision but not the fixed weighted expected semi-gradient. When the rollout distribution and all stopped coefficients are frozen at the current student, μ is the gradient of the corresponding step-local surrogate for F. Any importance-ratio truncation is applied before the token and response averages, and the factor w_i/m_i is applied after them; in that case the common expectation is for the same truncated surrogate.

A single mean over all valid tokens in the step would instead give a task more influence when it contributes either more responses or longer responses. This is the length-driven imbalance reported by Open-MOPD; response-level averaging followed by w_i/m_i weighting removes it by construction (Gao et al., 2026). Table A1 collects the notation.

3 GPAS

3.1 NOISE IN PRECONDITIONED COORDINATES

The variance criterion is exact for a step with a fixed diagonal preconditioner and provides a local allocation metric for optimizers with diagonal preconditioning. Let D be the pre-step diagonal preconditioner read from the optimizer state:

$$D = \begin{cases} I, & \text{for SGD,} \\ \operatorname{diag}\big((\sqrt{\widehat{v}_{\mathrm{pre}}} + \epsilon)^{-1}\big), & \text{for an Adam-type optimizer,} \end{cases}$$

where $\widehat{v}_{\mathrm{pre}}$ is the bias-corrected second-moment state at the start of the step. Noise in a coordinate affects the local update according to how strongly D amplifies it, so we measure noise after preconditioning and hold D fixed within a step.

The quantity to allocate is the variance of a single micro-batch gradient around its task mean. Unless noted otherwise, g_i and e_i refer to the top-k estimator used for training. Under the working model that fresh micro-batches are conditionally independent across draws and identically distributed within each task, with a common task mean and covariance, define task noise e_i and the accumulated-gradient variance $V(m)$ by

$$e_i = \mathbb{E}\big\|D(g_i - \mu_i)\big\|_2^2, \qquad V(m) = \mathbb{E}\big\|D(A - \mu)\big\|_2^2 = \sum_{i=1}^{M} \frac{w_i^2 e_i}{m_i}. \tag{3}$$

Thus the allocation enters the preconditioned variance only through the inverse-count terms. When μ is the gradient of the step-local surrogate, then for a fixed student state, preconditioner, and step size, the allocation also enters its one-step descent bound only through $V(m)$ (proof in Appendix A.2).

Minimizing this variance gives the count rule used by GPAS. Before enforcing integer bounds, the solution is

$$m_i \;\propto\; w_i \sqrt{e_i}. \tag{4}$$

This is Neyman allocation, the classical rule that samples each stratum in proportion to its standard deviation. We choose the common proportionality scale so that the clipped counts sum to G, iterating after any count reaches a bound, and then round by largest remainder.

Preconditioning matters because unpreconditioned and preconditioned noise can rank tasks differently. In a three-task example whose preconditioner reverses the noise ranking, counts from unpreconditioned noise raise the preconditioned variance to $2.3\times$ Uniform, whereas the preconditioned rule lowers it to $0.68\times$ (details in Appendix A.5).

3.2 WHERE THE TEACHER–STUDENT GAP ENTERS

The log ratio at a prefix has two distinct roles. Its student-probability weighted mean is the local reverse-KL gap, while its centered variation, together with the norm and direction of $\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})$, determines how noisy a sampled score-function term can be; the constant mean itself cancels against the mean-zero score function. Teachers at different distances from the student can therefore induce different within-position spreads, an empirical relation we test rather than assume.

For a precise diagnostic, view scoring in two stages: first fix the prompts and on-policy prefixes in a micro-batch, and then independently draw the token used to estimate each fixed-prefix score. Let $g^a_{i,\mathrm{diag}}$ denote the resulting diagnostic gradient for estimator $a \in \{\mathrm{samp}, \mathrm{top}k\}$, let $\bar{g}_i$ be their common conditional mean given the fixed trace, and let $\mu_i^{\mathrm{diag}} = \mathbb{E}[g^a_{i,\mathrm{diag}}]$. The law of total variance gives

$$e^a_{i,\mathrm{diag}} = e^{\mathrm{resp}}_{i,\mathrm{diag}} + e^{\mathrm{tok},a}_{i,\mathrm{diag}}, \qquad e^{\mathrm{resp}}_{i,\mathrm{diag}} = \mathbb{E}\big\|D(\bar{g}_i - \mu_i^{\mathrm{diag}})\big\|_2^2, \quad e^{\mathrm{tok},a}_{i,\mathrm{diag}} = \mathbb{E}\big\|D(g^a_{i,\mathrm{diag}} - \bar{g}_i)\big\|_2^2.$$

Here response-level noise comes from which prompts and rollout prefixes are observed, whereas token-level noise is the sum of the conditional within-position contributions from which scoring token is drawn at each fixed prefix (derivation in Appendix A.3). GPAS handles response-level noise by averaging more examples, whereas the retained-token component can be integrated directly at a fixed prefix.

The estimator in Section 2 replaces the random retained-set contribution by its exact fixed-prefix expectation. Its tail correction remains random, so the paired measurement in Section 4 checks the diagnostic variance change rather than presuming it. Online GPAS uses e_i from the actual top-k training gradients in Equation 3; the independent scoring-token protocol isolates the fixed-prefix mechanism, and its diagnostic components need not equal that online e_i. We test whether more distant teachers show a larger diagnostic token component and whether integrating the retained contribution predicts a narrower online noise spread and a changed allocation.

Open-MOPD uses the level of the gap as a budget signal and increases the weight of large-gap domains (Gao et al., 2026). Under a fixed objective, the log ratio already appears in the task gradient; its additional token-to-token variability is estimation noise, which we address with the estimator rather than by changing objective weights.

3.3 COST-AWARE MODE

Long-response tasks can take more time per micro-batch, so the allocation with the least variance per step need not have the least variance per hour. Let τ_i be the measured marginal time for the count-dependent rollout, teacher scoring, and backward work of one task-i micro-batch, and let C be the remaining fixed time for synchronization, the optimizer update, and logging. At a fixed student state, under the measured additive time model, the local time-normalized variance proxy is

$$J_C(m) = \Big(C + \sum_{i=1}^{M} m_i \tau_i\Big)\, V(m). \tag{5}$$

GPAS in its cost-aware mode enumerates the feasible integer count vectors and chooses the one with smallest J_C. In the unconstrained continuous relaxation with no fixed time, the optimal count scales as $w_i\sqrt{e_i/\tau_i}$; as fixed time dominates, it approaches the per-step rule (derivation in Appendix A.3).

3.4 ONLINE ESTIMATES AND IMPLEMENTATION

Every statistic for the next allocation comes from the step just completed. Let $\bar{g}_i$ now denote the sample mean of task i's observed micro-batch gradients in that step. Its preconditioned sample variance is

$$\widehat{e}_i = \frac{1}{m_i - 1} \sum_{s=1}^{m_i} \big\| D(g_{i,s} - \bar{g}_i) \big\|_2^2. \tag{6}$$

Under this working model, it is an unbiased estimate of one-micro-batch preconditioned noise, and the count lower bound guarantees that it exists every step. Timing estimates are smoothed online because response lengths make individual observations noisy.

To expose how much a count change can help, define the plug-in prediction $\widehat{V}(m) = \sum_i w_i^2 \widehat{e}_i / m_i$ and compare it with Uniform. The logged variance ratio is

$$H = \frac{\widehat{V}(m^{\mathrm{unif}})}{\widehat{V}(m)}, \tag{7}$$

and is bounded above by the count limits. The first step uses Uniform because no earlier statistics are available.

GPAS leaves the optimizer untouched: for SGD, D is the identity and no optimizer state is read; for Adam-type optimizers, D is read from the pre-step second-moment buffer. Changing the allocation changes the noise term in an Adam-type second-moment observation, as does any change in batch composition; SGD has no such state and is unaffected (details in Appendix A.4). Computing $\widehat{e}_i$ requires one parameter-sized mean buffer and elementwise reductions, but no additional forward or backward pass. The top-k estimator likewise reuses logits that the student and teacher already compute. Algorithm 1 summarizes one step.

1. Choose bounded integer counts m from the previous step's $\widehat{e}_i$ and, in the cost-aware mode, τ_i and C; use Uniform when no statistics are available.
2. For every task, draw fresh prompts and responses, obtain teacher scores for the student top-k tokens and for the sampled token when it is outside that set, and accumulate each micro-batch gradient with weight w_i/m_i.
3. Read the pre-update D from the optimizer state (the identity for SGD); from the same gradients and trace, compute $\widehat{e}_i$, update timing estimates, and log the diagnostic quantities.
4. Apply one optimizer update.

Algorithm 1: One optimizer step with GPAS.

4 Experimental Setup

4.1 Configuration

We distill four specialists into a Qwen3-1.7B student in non-thinking mode: the mathematics and instruction-following teachers are task-specific Qwen3-1.7B RL checkpoints, and the code and science teachers are Qwen3-4B in non-thinking mode. A micro-batch contains $b = 4$ prompts from one task, one response per prompt, and one backward pass. Each step accumulates $G = 16$ micro-batches, or 64 responses, with $m_{\min} = 2$ and $m_{\max} = 8$; the estimator uses the student's top $k = 16$ tokens. Every run lasts 500 steps, uses a nonrepeating stream of 16,000 prompts per task, and caps a response at 4,096 tokens. The fixed objective weights are proportional to the inverse initial task losses, and timing averages use decay 0.9. Uniform assigns four micro-batches to each task, so the count limits imply $H \leq 2$.

All training methods use AdamW with **[TODO: learning rate and schedule, β_1, β_2, ϵ, weight decay, gradient clipping, precision, and sharding]**, and each method uses one seed. A run reserves two GPUs: one 96-GB device trains the student and one 48-GB device generates rollouts and serves all four teachers. We evaluate 128 fixed held-out prompts per task every 50 steps and use 32 fresh micro-batches per task at early, middle, and late checkpoints for the paired variance protocol. Methods share the prompt order, sampling settings, optimizer, response cap, and checkpoint schedule. Table A2 gives the complete configuration and exact asset revisions.

4.2 Methods

The top-k estimator is the default unless a row explicitly says otherwise. **Uniform** uses the same count for every task. **GPAS (per step)** applies Equation 4 to preconditioned noise. **GPAS (per GPU hour)** is the same method in its cost-aware mode and minimizes Equation 5. **Counts from unpreconditioned noise** sets D to the identity only when computing counts; it is the SGD rule run under AdamW and isolates whether the optimizer's preconditioner matters. **Counts from loss gap** assigns counts in proportion to $w_i\bar{L}_i$, where $\bar{L}_i$ is the smoothed training-batch teacher loss, while retaining the fixed objective.

Two one-component ablations replace the top-k estimator by the usual sampled-token estimator under, respectively, Uniform counts and GPAS (per step) counts. These two rows and their top-k counterparts form the estimator by allocation comparison. **StdMOPD** uses the sampled-token estimator and averages all valid tokens in a step together; its effective task weights therefore follow token counts, so its objective differs from Equation 1. Every adaptive rule begins from a Uniform step, and all methods otherwise share the configuration above.

4.3 Measurements

Held-out teacher loss. Held-out teacher loss is the fixed-weight teacher loss on prompts that the training stream never sees. It is the primary training metric and is evaluated with a common sampling seed; method differences use paired bootstrap intervals over prompts, which measure evaluation uncertainty rather than variation over training seeds. Uniform's final held-out loss is the common target for compute comparisons.

Gap and token diagnostics. At every step and for every task, we log three scalars from the top-k scores. The gap level is the student-probability-weighted mean log ratio within the top-k set; the within-position spread is the corresponding weighted variance of that log ratio; and the tail mass is one minus the summed student probability of the set. The spread is a low-cost diagnostic rather than the gradient variance itself, because it omits the score vectors' directions and norms; the paired gradient measurement below tests whether it predicts token-level noise.

Paired gradient variance. At each selected checkpoint, we generate fresh micro-batches and first compute the actual top-k training gradient, whose sampled tail token comes from the rollout. These gradients supply $\widehat{e}_i$ for the held-out allocation comparison. We split the pool in half: one half estimates task noise and chooses an allocation, while the other scores that allocation using $V(m)$; swapping the halves and averaging removes the choice of split. Separately, after the response traces are fixed, the sampled-token and top-k diagnostic gradients share an independent scoring-token draw at every prefix. Their paired variance change isolates the fixed-prefix token mechanism but need not equal the online training-noise change (details in Appendix A.7).

Compute. GPU hours are full wall time times the two reserved GPUs, including waiting and synchronization. A trace assigns count-dependent rollout, scoring, and backward work to τ_i and the remaining fixed work to C, and we report the fit residual of this additive model. The overhead ratio is the fixed time per step divided by time spent on micro-batches; it caps how much a time-aware allocation can save (details in Appendix A.8).

Capability. We report MATH-500 greedy pass@1, a fixed LiveCodeBench pass@1 slice, IF-Bench strict accuracy, and GPQA-Diamond average@4 for the initial student, each teacher, and each method. For score s, the per-task normalized gain is $(s - s_{\text{init}})/(s_{\text{teacher}} - s_{\text{init}})$; we also report its average, mean response length, and truncation rate. **[TODO: Insert benchmark versions and complete decoding settings.]**

5 Results

5.1 Noise is heterogeneous, drifting, and coordinate dependent

[TODO: Insert Figure 2 from the four-teacher logs: (a) preconditioned and unpreconditioned noise over training; (b) counts chosen by GPAS in its per-step and per-GPU-hour modes; (c) stacked response-level and removable token-level noise by task, aligned with gap level and teacher origin.]

Figure 2: Task-gradient noise, allocation, and its token component. **[TODO: State the cross-task range, drift, coordinate-ranking disagreement, and token share for distant versus same-origin teachers.]**

Table 1: Held-out preconditioned top-k gradient variance relative to Uniform. Each allocation is selected on one half of fresh micro-batches and evaluated on the other. **[TODO: Fill the checkpoint values and the relative error of $\widehat{e}_i$ across halves.]**

Allocation	Early	Middle	Late
Uniform	1.00	1.00	1.00
Counts from unpreconditioned noise			
Counts from loss gap			
GPAS (per step)			
GPAS (per GPU hour)			

[TODO: A single-micro-batch noise estimate spans X across tasks and each task drifts by up to Y over training; the preconditioned and unpreconditioned rankings disagree in Z of steps. Report count-bound occupancy and the median and quartiles of H.] The controlled calculation already shows why the coordinate choice can matter: the unpreconditioned rule reaches $2.3\times$ Uniform's preconditioned variance, whereas GPAS reaches $0.68\times$.

5.2 Teacher–student gap appears as token-level noise

Figure 2c separates response-level noise from the fixed-prefix token component targeted by the top-k estimator. **[TODO: Report the token-noise fraction for distant and same-origin teachers, its association with gap level and within-position spread, the tail mass, and the reduction in the cross-task noise ratio and resulting count change.]** This comparison treats teacher distance as a measured correlate, not as an assumed monotone law.

5.3 Analytic removal and sample allocation are complementary

Table 2: Four-teacher capability and efficiency. StdMOPD optimizes a different objective, so its held-out F and compute-to-target entries are not comparable. **[TODO: Populate from evaluation outputs and timing logs; report every task-level difference from Uniform.]**

	Math	Code	IF	Science	Norm. avg.	Final F	GPU h to target
Initial student							
Teachers							
StdMOPD							
Uniform							
GPAS (per step)							
GPAS (per GPU hour)							
Counts from unpreconditioned noise							
Counts from loss gap							
Sampled-token estimator (Uniform)							
Sampled-token estimator (GPAS per step)							

Table 3: Matched-step 2×2 ablation of the gradient estimator and count allocation. **[TODO: Fill held-out teacher loss and paired differences from the four runs.]**

	Uniform counts	GPAS (per step) counts
Sampled-token estimator		
Top-k estimator		

[TODO: At matched steps, integrating the retained-token contribution improves held-out loss by X, allocating the residual noise improves it by Y, and the combination improves it by Z. Once that contribution is integrated, counts from loss gap provide no additional advantage over the noise rule.]

5.4 THE OVERHEAD RATIO BOUNDS THE GAIN FROM TIME-AWARE ALLOCATION

[TODO: Insert Figure 3: (a) held-out weighted teacher loss versus optimizer steps for the matched estimator–allocation comparison; (b) the same fixed objective versus GPU hours for Uniform and GPAS in its per-step and per-GPU-hour modes. Annotate the measured overhead ratio and the time to Uniform's final loss.]

Figure 3: Matched-step learning curves and compute efficiency under the measured system overhead. **[TODO: State the overhead ratio and GPU-hour saving at the common loss target.]**

[TODO: At overhead ratio R, GPAS (per GPU hour) reaches Uniform's final loss in S fewer GPU hours. Relate the observed saving to the upper limit implied by fixed step time and report the additive time model's error.]

6 RELATED WORK

Task weighting and data allocation. Multi-task methods tune loss weights, alter gradient directions, or adapt data schedules (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018), and language-model training adapts domain, rollout, or prompt distributions from learning progress (Albalak et al., 2023; Wang et al., 2025; Zheng et al., 2025; Gao et al., 2025). These methods change the objective or the data distribution. GPAS keeps both fixed and changes only the number of micro-batches used to estimate each task gradient; because every task appears in every step, it observes within-task gradient noise for all tasks rather than for a sampled subset.

Multi-teacher on-policy distillation. MOPD routes student rollouts to task teachers and trains a shared student with token-level teacher feedback (Ma et al., 2026). Open-MOPD studies token-share balancing and gap-following, and also handles stale rewards (Gao et al., 2026); its token-share balance holds here by construction through response averaging and w_i/m_i weighting. We use one update per rollout and do not address reuse. D^3-MOPD adapts the domain mixture from the teacher-loss gap and its descent velocity (Sun et al., 2026), which changes the objective weights; GPAS changes the precision of each task-gradient estimate instead. Teacher mixtures can still produce transfer and task trade-offs (Li et al., 2026), which is why we report capability per task.

Top-k distillation estimators. Three estimators are commonly described as top-k distillation. The first restricts reverse KL to the student's top-k tokens and renormalizes; it drops the mass outside the set and therefore optimizes a different objective, and MOPD reports that it diverges earlier than the sampled-token form under a large teacher–student gap (Ma et al., 2026). The second, used by Open-MOPD, sums top-k log ratios into a scalar reward that multiplies the sampled token's score function. This reward is smoother than a single log ratio, but randomness from the sampled token remains. The third is the sampled-token log ratio itself. Our estimator computes that third form's top-k contribution exactly and lets a sampled token cover the remaining mass, preserving its fixed-prefix expectation while removing the retained-set sampling component.

Stratified allocation for optimization. Gradient-norm sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Neyman allocation assigns samples in proportion to within-stratum standard deviation, and unequal sampling costs give the square-root cost correction used here (Neyman, 1934; Cochran, 1977; Mohri et al., 2026). The right variance geometry depends on the optimizer: Euclidean coordinates suit SGD, whereas adaptive optimizers precondition coordinates (Lahire, 2023). GPAS applies Neyman allocation to fixed-size MOPD micro-batches under the preconditioner of whichever optimizer is used.

7 Limitations and Conclusion

The study covers one student scale, four tasks, one seed per method, and one system layout. GPAS is a local rule: its analysis treats micro-batches within a step as conditionally independent and identically distributed within task, holds the pre-step preconditioner fixed while measuring noise, and its attainable variance reduction is bounded by the count limits. The additive step-time model is fitted to one layout; systems with parallel critical paths require their own measured time model. The method needs at least two micro-batches per task per step, so settings with many more tasks than micro-batches need a task-subsampling variant. The variance derivation treats fresh micro-batches as conditionally i.i.d.; the fixed shuffled prompt stream is read without replacement and therefore has a finite-population covariance correction. Held-out splits and online estimates measure the realized setting directly.

Each rollout batch is used for one update; staleness caused by reusing rollouts across updates is outside our scope. The top-k estimator also requires the teacher to score the student's retained tokens and the sampled token when it lies outside that set. This is natural when the teacher is available as a prefill scoring service, but it may not be available through an API that exposes only the sampled-token score.

GPAS fixes the objective weights and adapts only how many micro-batches each task contributes, so the allocation changes gradient variance and nothing else. Its per-step mode follows Neyman allocation in preconditioned coordinates, and its cost-aware mode also uses measured time; both reuse statistics produced by MOPD training. **[TODO: Conclude with the four empirical laws in one sentence: noise is heterogeneous, drifting, and coordinate dependent; teacher distance predicts a removable token component; analytic removal and allocation are complementary; and the overhead ratio bounds the gain per GPU hour.]**

AI use statement

Generative AI tools assisted with language editing and manuscript organization. The authors remain responsible for reviewing the text, claims, proofs, code, and experimental results before submission.

Ethics statement

The study evaluates optimization methods for existing language models with reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect capability scores. The final artifact will record data provenance, invalid outputs, and verifier failures to support careful interpretation.

Reproducibility statement

Section 4 specifies the micro-batch composition, fixed objective weights, integer allocation, resource accounting, baselines, held-out evaluation, and matched prompt streams. The current artifact includes the controlled-check code, seeds, and source data for the controlled figure. **[TODO: Before submission, add the frozen run configuration, model and tokenizer revisions, initial-loss weights, prompt streams, optimizer and allocation states, micro-batch counts, noise and timing logs, per-prompt evaluation outputs, and source data for every empirical figure and table.]**

References

Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL https://arxiv.org/abs/1511.06481.

Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL https://arxiv.org/abs/2312.02406.

Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of*

the 35th International Conference on Machine Learning, pp. 794–803, 2018. URL https://proceedings.mlr.press/v80/chen18a.html.

William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.

Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL https://arxiv.org/abs/2608.19098.

Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training, 2025. URL https://arxiv.org/abs/2510.01135.

Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL https://proceedings.mlr.press/v80/katharopoulos18a.html.

Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL https://opt-ml.org/papers/2023/paper20.pdf.

Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL https://arxiv.org/abs/2608.16647.

Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL https://arxiv.org/abs/2110.14048.

Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL https://arxiv.org/abs/2606.30406.

Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL https://arxiv.org/abs/2604.28020.

Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.

Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL https://arxiv.org/abs/1810.04650.

Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D^3-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL https://arxiv.org/abs/2608.24987.

Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL https://arxiv.org/abs/2504.09710.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL https://arxiv.org/abs/2001.06782.

Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts, 2025. URL https://arxiv.org/abs/2506.02177.

A Supplementary Tables, Derivations, and Details

A.1 Notation and configuration

Table A1: Notation and the values used in the four-teacher study.

Symbol	Meaning	Value in this study
M	number of tasks and teachers	4
w_i	fixed objective weight	inverse initial loss, normalized
G	micro-batches per optimizer step	16
b	prompts per micro-batch, one response each	4
m_i	task-i micro-batch count	$2 \le m_i \le 8$
$g_{i,s}$	unweighted micro-batch gradient	from the backward pass
D	diagonal preconditioner; identity for SGD	pre-step optimizer state
e_i, $\widehat{e}_i$	preconditioned one-micro-batch noise; its estimate	estimated each step
τ_i, C	time per task-i micro-batch; fixed time per step	exponential averages
H	predicted variance ratio relative to Uniform	bounded by count limits

Table A2: Four-teacher MOPD configuration.

Component	Configuration
Student	Qwen3-1.7B (Qwen Team, 2025) in non-thinking mode; **[TODO: exact frozen revision and initial scores]**
Teachers	Math and instruction following use task-specific Qwen3-1.7B RL checkpoints; code and science use one frozen Qwen3-4B weight set through separate task endpoints. All run in non-thinking mode and have byte-identical tokenizer files; **[TODO: exact identifiers and checkpoint steps, RL recipe, and teacher scores]**
Training prompts	16,000 unique prompts per task, shuffled once and read without replacement; **[TODO: dataset names, versions, licenses, and filters]**
Rollout and loss	One response per prompt; one update per rollout batch; top-k reverse-KL estimator with $k = 16$, averaged over valid tokens and then responses; teacher scores the student top-k union the sampled token; maximum response length 4,096, truncated responses kept; **[TODO: temperature, ratio-truncation rule, and observed truncation rate by task]**
Step and budget	$b = 4$ prompts per micro-batch, $G = 16$ micro-batches per step, 500 steps, and 32,000 attempted responses; $2 \le m_i \le 8$
Optimization	AdamW; inverse-initial-loss objective weights fixed before training; timing-average decay 0.9; **[TODO: learning rate and schedule, β_1, β_2, ϵ, weight decay, gradient clipping, precision, and sharding]**
Evaluation	Held-out teacher loss on 128 fixed prompts per task from checkpoints every 50 steps; MATH-500, LiveCodeBench slice, IFBench, GPQA-Diamond; **[TODO: benchmark versions and decoding settings]**
Seeds	1 per method
System	Two GPUs per run: one 96 GB GPU trains the student, one 48 GB GPU serves rollouts and holds the four resident teachers; **[TODO: exact GPU models, inference engine, software revisions, and GPU hours per run]**

A.2 One-step variance criterion

Condition on the pre-step history $\mathcal{F}$ and let the positive diagonal matrix D be fixed before the current micro-batches are drawn. Suppose the conditional mean $\mu = \mathbb{E}[A \mid \mathcal{F}]$ is the gradient of an L-smooth local surrogate $F_{\mathcal{F}}$. Smoothness gives, for the local frozen-preconditioner update $\theta' = \theta - \eta DA$,

$$\begin{aligned} F_{\mathcal{F}}(\theta') &\le F_{\mathcal{F}}(\theta) - \eta \mu^\top DA + \frac{\eta^2 L}{2} \|DA\|_2^2, \\ \mathbb{E}[F_{\mathcal{F}}(\theta') \mid \mathcal{F}] &\le F_{\mathcal{F}}(\theta) - \eta \mu^\top D\mu + \frac{\eta^2 L}{2} \left(\|D\mu\|_2^2 + V(m) \right). \end{aligned} \tag{8}$$

The second line uses $\mathbb{E}[A \mid \mathcal{F}] = \mu$ and the bias–variance identity. For a fixed student state, D, and step size, only $V(m)$ depends on the allocation. This local bound motivates the allocation score; an Adam-type optimizer with momentum, a current-gradient second-moment update, and gradient clipping is not identical to the frozen-preconditioner step, and the experiment measures the end-to-end effect with its conventional optimizer.

A.3 Gradient variance, its estimate, and the allocation rules

Micro-batches that are conditionally independent across draws and identically distributed within each task, with a common task mean and covariance, give

$$\mathbb{E}\|D(A-\mu)\|_2^2 = \sum_i w_i^2\, \mathbb{E}\|D(\bar{g}_i - \mu_i)\|_2^2 = \sum_i \frac{w_i^2}{m_i}\, \mathbb{E}\|D(g_i - \mu_i)\|_2^2, \tag{9}$$

which is Equation 3. Within a step, $\sum_s \|D(g_{i,s}-\bar{g}_i)\|_2^2 = \sum_s \|Dg_{i,s}\|_2^2 - m_i\|D\bar{g}_i\|_2^2$, so Equation 6 is the usual unbiased sample variance summed over the preconditioned coordinates and is defined whenever $m_i \geq 2$.

In the positive-real relaxation without box constraints, write $a_i = w_i\sqrt{e_i}$. With equal micro-batch cost, the Cauchy–Schwarz inequality

$$\Big(\sum_i m_i\Big)\Big(\sum_i \frac{a_i^2}{m_i}\Big) \geq \Big(\sum_i a_i\Big)^2 \tag{10}$$

holds with equality at $m_i \propto a_i$, which is GPAS. With task-dependent cost and zero fixed overhead,

$$\Big(\sum_i m_i\tau_i\Big)\Big(\sum_i \frac{a_i^2}{m_i}\Big) \geq \Big(\sum_i a_i\sqrt{\tau_i}\Big)^2, \tag{11}$$

with equality at $m_i \propto a_i/\sqrt{\tau_i}$. With positive fixed time, the cost-aware mode evaluates Equation 5 over the feasible integer counts. When fixed time dominates the count-dependent term, that objective is a constant multiple of $V(m)$ to first order and its minimizer approaches the per-step rule.

For the scoring-noise decomposition in Section 3.2, let $\mathcal{R}_i$ contain the sampled prompts, complete rollouts, observed prefixes, and response lengths. For $a \in \{\mathrm{samp}, \mathrm{top}k\}$, hold $\mathcal{R}_i$ fixed and let G_i^a be the micro-batch gradient obtained by independent scoring-token draws at those prefixes. Conditional unbiasedness gives the common mean $\bar{G}_i = \mathbb{E}[G_i^a \mid \mathcal{R}_i]$; define its unconditional value as $\mu_i^{\mathrm{diag}} = \mathbb{E}[G_i^a]$. Orthogonality of a conditional-mean residual gives

$$\begin{aligned}\mathbb{E}\|D(G_i^a - \mu_i^{\mathrm{diag}})\|_2^2 = \mathbb{E}\|D(\bar{G}_i - \mu_i^{\mathrm{diag}})\|_2^2 \\ + \mathbb{E}\|D(G_i^a - \bar{G}_i)\|_2^2.\end{aligned}$$

These three quantities are, respectively, $e^a_{i,\mathrm{diag}}$, $e^{\mathrm{resp}}_{i,\mathrm{diag}}$, and $e^{\mathrm{tok},a}_{i,\mathrm{diag}}$ in the main text. Conditional on $\mathcal{R}_i$, response lengths and averaging coefficients are fixed; with independent scoring draws, the last term is the sum of the per-position conditional variances after those coefficients and D are applied. The top-k estimator integrates the retained-token terms in this second stage and leaves a sampled tail residual.

A.4 Allocation and Adam-type second moments

SGD has no second-moment state and is unaffected by this issue. For an Adam-type optimizer and a fixed allocation, assume task-i micro-batch gradients are independent with mean h_i and coordinate-wise variance σ_i^2. Equation 2 gives the componentwise identity

$$\mathbb{E}[A \odot A] = \Big(\sum_i w_i h_i\Big) \odot \Big(\sum_i w_i h_i\Big) + \sum_i \frac{w_i^2}{m_i}\sigma_i^2. \tag{12}$$

Thus the current noise contribution to the second-moment observation depends on the allocation, as it does whenever batch composition changes. Under the study's count limits, each task's contribution lies between $0.5\times$ and $2\times$ its Uniform value. The experiments use conventional AdamW and test the complete parameter update rather than inferring it from this observation alone.

For completeness, an allocation-independent observation can define a distinct optimizer design:

$$U = \sum_{i=1}^{M} \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s} \odot g_{i,s}, \qquad \mathbb{E}[U/G] = \frac{1}{G} \sum_i w_i \big(h_i \odot h_i + \sigma_i^2\big), \tag{13}$$

Its expectation does not depend on the counts. At proportional allocation its noise term has the same scale as the conventional observation, but its signal term differs, so it is not an interchangeable Adam target and is not used in our training runs. In the controlled check below, reallocating from Uniform to GPAS changes the conventional observation by 0.543 in relative norm (calculated 0.542) and the U/G observation by 0.0009 (calculated zero).

A.5 Controlled calculation

This illustrative calculation has its own task count and count bounds; they are not the experimental configuration. It uses three Gaussian micro-batch gradients in three coordinates and a diagonal preconditioner that reverses their noise ranking. Each step contains $G = 16$ micro-batches with $2 \le m_i \le 12$. Bounded rounding gives counts $(6, 5, 5)$ for Uniform, $(11, 3, 2)$ for counts from unpreconditioned noise, $(3, 3, 10)$ for GPAS, and $(3, 5, 8)$ for GPAS in the zero-fixed-time cost-aware mode. All methods use the same fixed weights and common random numbers over $20{,}000$ trials. GPAS attains $0.679\times$ Uniform's preconditioned variance (calculated $0.681\times$), the unpreconditioned count rule reaches $2.316\times$, and the cost-aware mode reaches a time–variance product of $0.857\times$ Uniform (calculated $0.854\times$). The second-moment panel uses $200{,}000$ simulated steps.

Table A3: Numerical values for the synthetic three-task, zero-fixed-time calculation behind Figure 4, reported to four decimal places. Variance and predicted time–variance products are normalized by Uniform. Monte Carlo estimates use $20{,}000$ common-random-number trials with seed 20260902.

		Preconditioned variance		Time–variance	
Method	Counts	Calculated	Monte Carlo	Calculated	Monte Carlo
Uniform	$(6, 5, 5)$	1.0000	1.0000	1.0000	1.0000
Counts from unpreconditioned noise	$(11, 3, 2)$	2.3224	2.3157	2.0901	2.0842
GPAS (per step)	$(3, 3, 10)$	0.6808	0.6793	0.9153	0.9133
GPAS (per GPU hour)	$(3, 5, 8)$	0.7321	0.7345	0.8542	0.8569

Table A4: Relative norm change in optimizer observations when reallocating from Uniform counts $(6, 5, 5)$ to GPAS counts $(3, 3, 10)$. Monte Carlo estimates use $200{,}000$ draws with seed 20260903.

Observation	Calculated change	Monte Carlo change
First moment	0.0000	0.0021
Conventional second moment $A \odot A$	0.5420	0.5428
Alternative second moment U/G	0.0000	0.0009

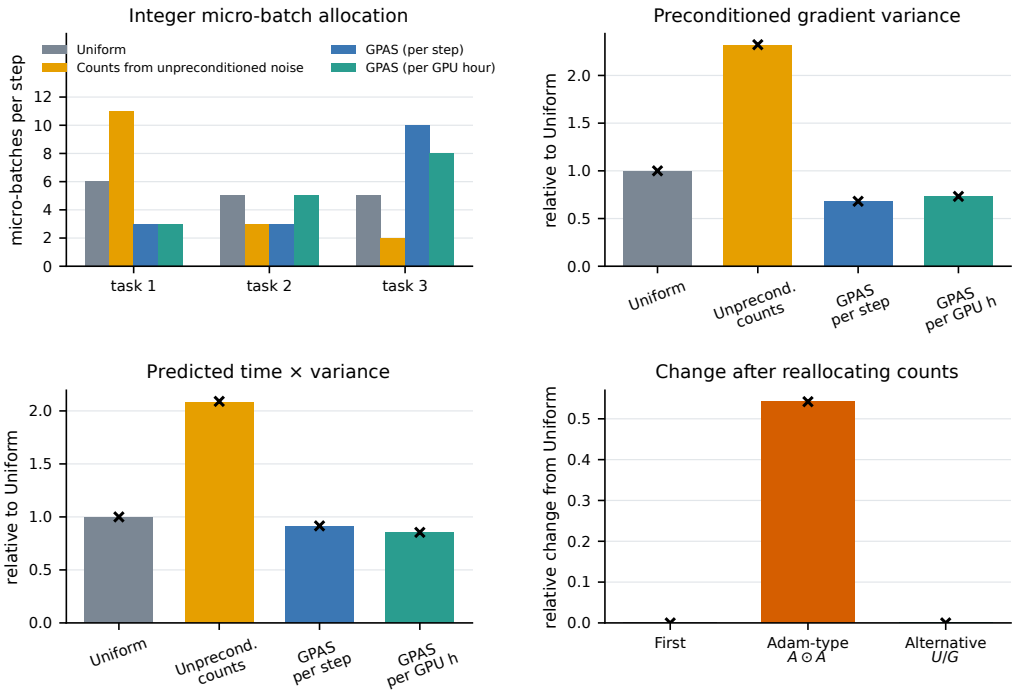

Figure 4: Synthetic three-task controlled checks with zero fixed time. Bars are Monte Carlo estimates and crosses are calculated values. The per-GPU-hour label denotes the predicted time–variance product under the synthetic task costs, not measured GPU hours. The last panel compares second-moment observations after an integer reallocation.

A.6 Conditional unbiasedness of the top-k estimator

Fix a prompt x and student-generated response prefix $y_{<t}$, and use the notation of Section 2. Linearity of expectation separates the exact retained set from the sampled tail:

$$\begin{aligned}
&\mathbb{E}_{y_t}\left[\widehat{h}_i^{\mathrm{topk}}(x, y_{<t}, y_t) \mid x, y_{<t}\right] \\
&= \sum_{a \in \mathcal{K}_k(x, y_{<t})} \pi_\theta(a \mid x, y_{<t}) r_{i,a}(x, y_{<t}) + \sum_{a \notin \mathcal{K}_k(x, y_{<t})} \pi_\theta(a \mid x, y_{<t}) r_{i,a}(x, y_{<t}) \\
&= \sum_a \pi_\theta(a \mid x, y_{<t}) r_{i,a}(x, y_{<t}) \\
&= \mathbb{E}_{y_t}\left[\widehat{h}_i^{\mathrm{samp}}(x, y_{<t}, y_t) \mid x, y_{<t}\right].
\end{aligned} \tag{14}$$

Thus the estimator is conditionally unbiased at every fixed prefix. In an automatic-differentiation implementation, the top-k membership and the coefficients $\pi_\theta(a \mid x, y_{<t})[\log \pi_\theta(a \mid x, y_{<t}) - \log \pi_{T_i}(a \mid x, y_{<t})]$ are detached; otherwise extra derivative terms would be introduced. The exact term removes randomness among retained tokens, but the tail indicator and tail-token identity remain random, so the net variance change is measured rather than assumed. The equality above is position-level; the paired protocol fixes rollouts, prefixes, and response lengths before it compares scoring estimators. If importance-ratio truncation is used, the same proof applies to the resulting truncated surrogate rather than to the untruncated reverse KL.

A.7 Held-out gradient variance protocol

At the early, middle, and late Uniform checkpoints, each task supplies 32 fresh micro-batches from held-out prompts. We first compute each actual top-k training gradient, using the rollout token for its tail correction; these gradients supply the held-out $\widehat{e}_i$ used to select and score allocations.

Separately, after fixing the sampled responses, prefixes, and lengths, we independently draw one scoring token from $q(\cdot \mid z)$ at every prefix and use the same draw for the sampled estimator and the top-k tail correction. For each diagnostic version we record $\|Dg_{i,s}\|_2^2$ and the squared norm of the preconditioned half-sample mean, using D from the checkpoint's pre-step AdamW state. The paired diagnostic difference

$$\widehat{\Delta}_{i,\mathrm{diag}} = \widehat{e}^{\,\mathrm{samp}}_{i,\mathrm{diag}} - \widehat{e}^{\,\mathrm{top}k}_{i,\mathrm{diag}} \tag{15}$$

is the operational estimate of the change in the fixed-prefix token component. We report its sign and uncertainty; it is an empirical variance change, not a guarantee that the top-k estimator lowers total trajectory variance.

Half $\mathcal{A}$ estimates actual top-k training noise and supplies each count rule with an allocation $m^{\mathcal{A}}$; counts from loss gap use the checkpoint's held-out losses. For the per-GPU-hour row, τ_i and C are the training-time exponential averages frozen immediately before that checkpoint, rather than timings from the fresh diagnostic pool. Half $\mathcal{B}$ scores that allocation with actual top-k gradients by $\sum_i w_i^2 \widehat{e}_i^{\mathcal{B}} / m_i^{\mathcal{A}}$ relative to Uniform. We exchange the halves and average. We also compute the relative error between half-sample noise estimates and repeat the estimate on subsets matching the online count range. Only scalar norms and the three gap diagnostics are stored.

A.8 System timing and logs

For each task in a step, the log records rollout, scoring, backward, and optimizer time, peak memory, attempted responses, and valid and generated tokens. We fit the additive model $C + \sum_i m_i \tau_i$ to measured step times: τ_i absorbs all task-i work that changes with its count, including marginal generation, teacher scoring, and backward work, while C contains only count-independent synchronization, optimizer, and logging time. We report fit residuals and do not infer that a response-length cap makes rollout time constant. GPU hours are wall time times the two reserved GPUs, including all waiting. Checkpoints store the allocation, noise estimates, timing averages, and prompt positions, and a resume test reproduces the next allocation and parameter update. A layout with a parallel critical path should replace the additive model by its fitted step-time function; the fixed objective and per-step GPAS rule are unchanged.

A.9 Full task-level trajectories

[TODO: Insert per-task held-out and training losses, prompt consumption, response length and truncation, count-bound occupancy, and step-time quantiles from the released logs.]

Figure 5: Task-level training and system traces. **[TODO: Replace after the four-teacher runs finish.]**